CS 683- Assignment 1- Task-1 Report

24b1078, 24b1020, 24b1046, 24b1082

September 6, 2026

1 Methodology

Five optimized implementations of 2D convolution, `conv_reorder`, `conv_unroll`, `conv_tile`, `conv_simd` (256-bit AVX2), and `conv_optimized` (tiling + SIMD + manual unrolling combined), were benchmarked against the `conv_naive` reference on identical inputs. Each configuration (matrix size $\times$ kernel size $\times$ stage) was run 10 times with different random seeds, and the reported speedup is the mean across those 10 runs. Matrix sizes ranged from 128×128 to 4096×4096 (all multiples of 8), and kernel sizes $K = 3$ and $K = 5$ were tested.

2 Results

Table 1 and Table 2 give the mean speedup of each technique over `conv_naive` for every matrix size, at $K = 3$ and $K = 5$ respectively.

Table 1: Mean speedup vs. naive, $K = 3$

Size	reorder	unroll	tile	simd	optimized
128	1.51	3.72	1.16	7.33	11.63
256	1.69	3.85	1.26	9.50	14.14
512	1.70	3.46	1.13	8.80	12.47
1024	1.56	3.71	1.22	8.99	13.27
1536	1.18	3.81	1.18	9.23	11.58
2048	1.11	3.66	1.12	7.82	10.26
3072	1.10	3.67	1.19	8.16	10.01
4096	1.10	3.71	1.18	7.55	10.25

Table 2: Mean speedup vs. naive, $K = 5$

Size	reorder	unroll	tile	simd	optimized
128	1.53	3.79	1.02	5.18	14.30
256	1.76	4.15	1.27	6.39	18.20
512	1.87	4.33	1.32	6.38	18.07
1024	1.76	3.91	1.17	5.64	15.78
1536	1.23	3.81	1.10	5.62	13.85
2048	1.16	3.84	1.15	5.59	14.66
3072	1.19	3.79	1.15	5.44	14.87
4096	1.14	3.69	1.11	5.52	13.99

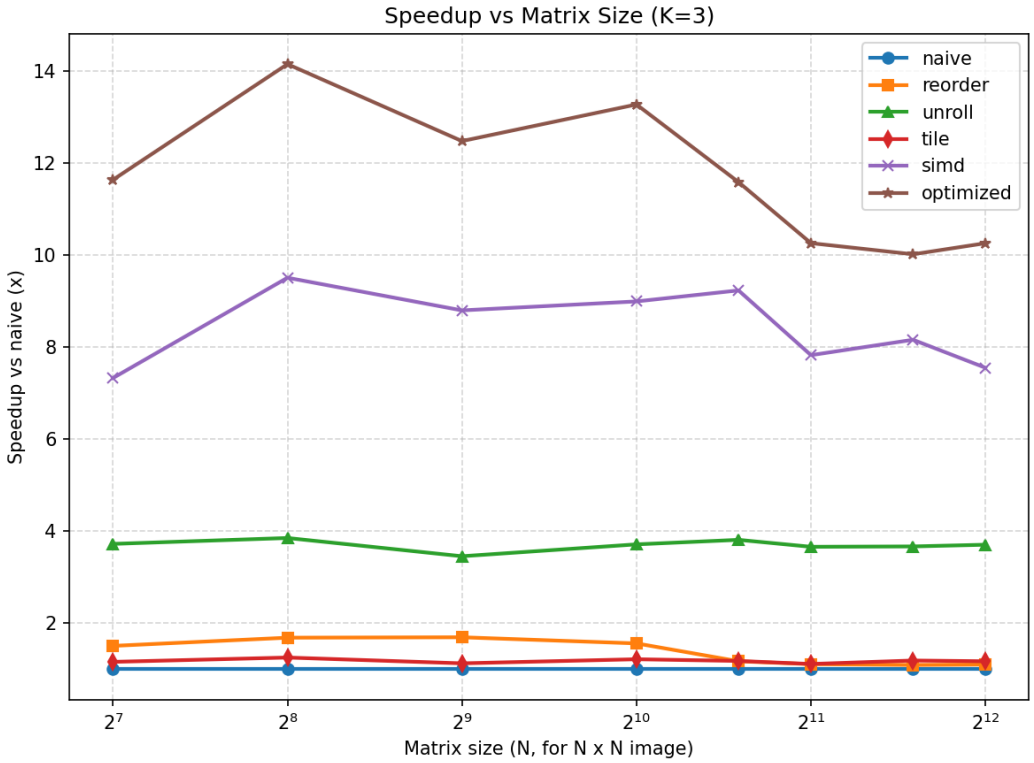

Figure 1: Speedup vs. matrix size, $K = 3$, all techniques.

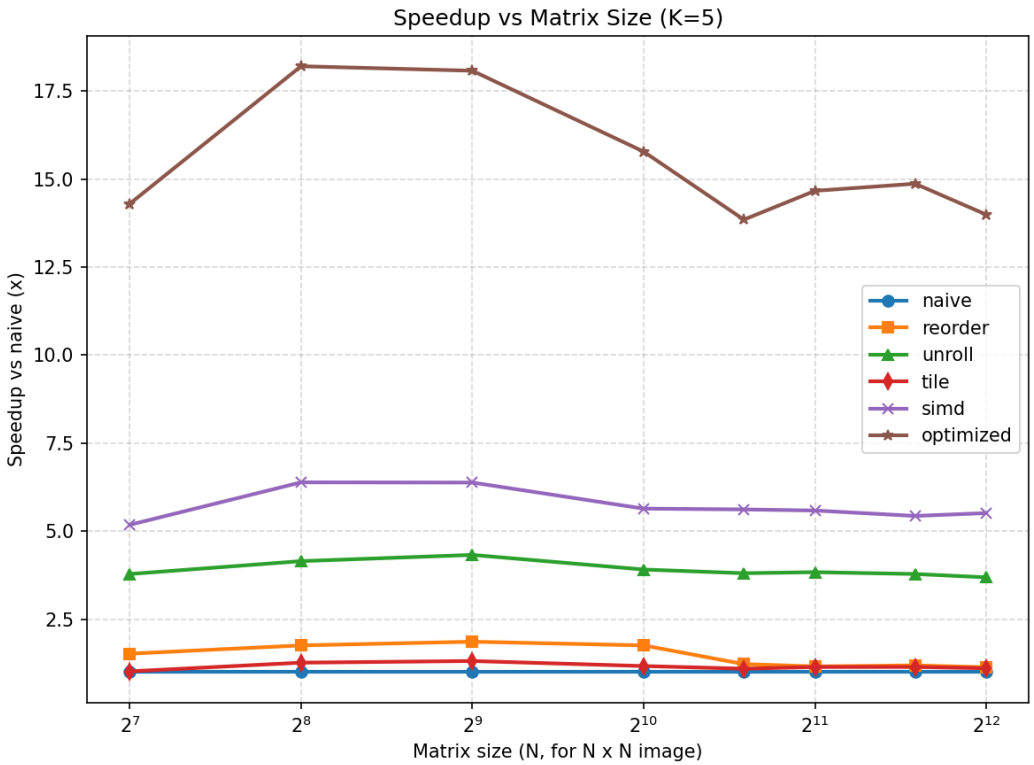

Figure 2: Speedup vs. matrix size, $K = 5$, all techniques.

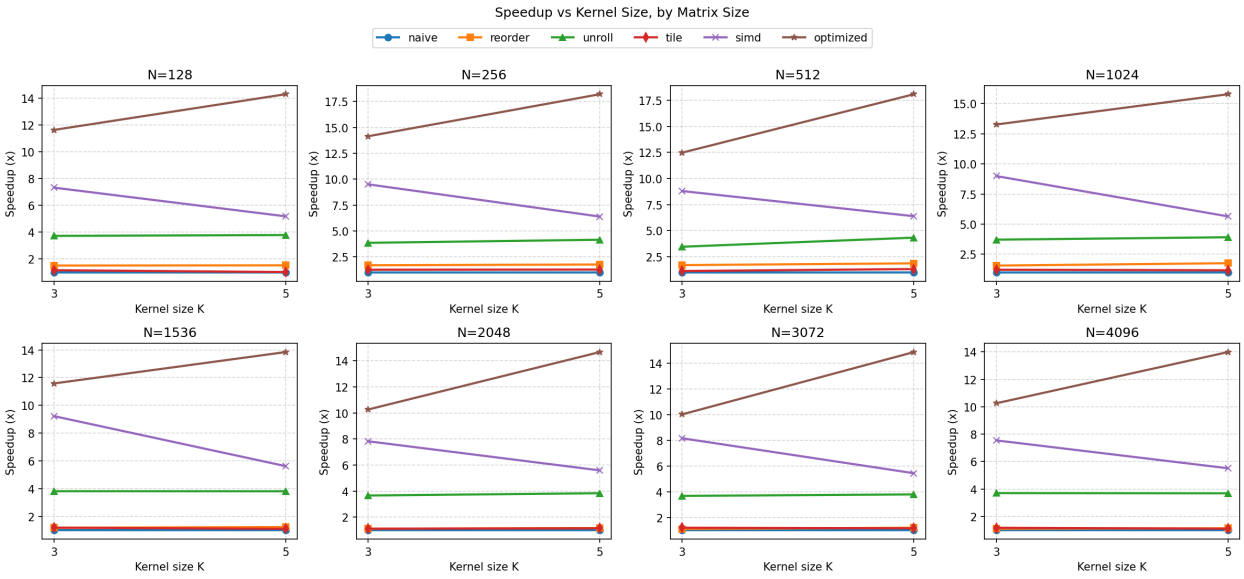

Figure 3: Speedup vs. kernel size ($K = 3$ vs. $K = 5$), faceted by matrix size, all techniques.

3 Discussion: Justifying the Observed Gains and Losses

Loop reordering (`conv_reorder`)

Reordering hoists the kernel loops outward so the innermost loop streams unit-stride over output columns, which should improve spatial locality. In practice the gain is small (1.1–1.9×) and it *shrinks* as matrix size grows (from ∼1.7× at $N \leq 512$ down to ∼1.1× by $N = 2048$). This is because reordering alone still performs K^2 full passes over the output image; once the image exceeds cache capacity, each pass re-fetches data from DRAM regardless of the loop order, so the locality benefit that reordering provides at small sizes disappears at large sizes.

Loop unrolling (`conv_unroll`)

Unrolling exposes independent work to hide floating-point latency and gives a consistent 3.4–4.3× speedup that is essentially *flat* across all matrix sizes, for both $K = 3$ and $K = 5$. This flatness indicates unrolling addresses instruction-level parallelism (pipeline stalls from dependent FMA chains), a compute-side bottleneck that does not depend on whether the working set fits in cache, unlike reordering or tiling, which target memory locality.

Cache tiling (`conv_tile`)

Tiling alone yields only a modest 1.0–1.3× speedup, smaller than what might be expected from the assignment's emphasis on cache blocking. This is because `conv_tile` in isolation still runs scalar arithmetic, it improves data locality but does not reduce the raw instruction count or exploit vector units, so its benefit is capped by how memory-bound the scalar loop was to begin with. Its real value is revealed only once combined with SIMD (see `conv_optimized` below).

SIMD vectorization (`conv_simd`)

SIMD gives the largest single-technique speedup (5.2–9.5×), since each AVX2 instruction processes 8 output pixels via a fused multiply-add. Two trends are notable:

- **Vs. matrix size**: speedup peaks around $N = 256$–1536 where the working set still fits in cache, then declines by ∼15–20% at $N \geq 2048$ as the image exceeds L2/L3 capacity and the vector unit becomes memory-starved, SIMD reduces instruction count, but cannot reduce time spent waiting on DRAM bandwidth.
- **Vs. kernel size**: speedup at $K = 5$ is consistently *lower* than at $K = 3$ (e.g. 7.82× → 5.59× at $N = 2048$). Increasing K from 3 to 5 raises taps-per-pixel from 9 to 25 (2.8× more FMA work), which is a compute increase that scales naive and SIMD execution time by similar factors, so the *ratio* between them narrows even though SIMD's absolute time is still much lower than naive's.

Combined optimization (`conv_optimized`)

The fully combined stage (tiling + SIMD + manual 2-wide unrolling) dominates every individual technique at every configuration tested (10–18×). Its two trends are essentially the mirror image of plain SIMD's:

- **Vs. matrix size**: it declines more gently than plain `simd` as N grows, because tiling keeps the working set cache-resident regardless of the overall image size, so the vector unit stays fed even at $N = 4096$.
- **Vs. kernel size**: speedup *increases* with K (e.g. $10.26\times \rightarrow 14.66\times$ at $N = 2048$), the opposite of plain `simd`. Because tiling serves the extra taps introduced by a larger kernel from cache rather than DRAM, the additional compute from $K = 5$ is handled efficiently by the vector unit while `conv_naive` pays the full memory-and-compute cost of every extra tap, so the relative gap widens rather than narrows.

This confirms that tiling and SIMD are complementary: SIMD reduces per-tap compute cost, while tiling ensures that cost is paid against cache rather than DRAM, and the combination scales better with both image size and kernel size than either technique alone.